PyQt6 — Шпаргалка

Основы для разработки десктоп-приложений (GUI + 3D графика)

Python 3.10+ · PyQt6 6.x · OpenGL через PyOpenGL

1. Установка и структура проекта

1.1 Установка

```
pip install PyQt6 PyQt6-Qt6 PyQt6-sip
pip install PyOpenGL PyOpenGL_accelerate
pip install numpy          # обязателен для буферов OpenGL
pip install ifcopenshell   # для работы с .ifc файлами
```

1.2 Минимальный проект

```
# main.py
import sys
from PyQt6.QtWidgets import QApplication, QMainWindow

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle('IFC Viewer')
        self.resize(1280, 720)

if __name__ == '__main__':
    app = QApplication(sys.argv)
    window = MainWindow()
    window.show()
    sys.exit(app.exec())   # запуск event loop
```

1.3 Структура типичного проекта

```
project/
├── main.py               # точка входа
├── mainwindow.py         # MainWindow
├── gl_widget.py          # QOpenGLWidget — 3D вьюпорт
├── ifc_loader.py         # Worker для загрузки .ifc
├── shaders/
│   ├── vertex.glsl
│   └── fragment.glsl
└── resources/
    └── icons/
```

> *В PyQt6 нет .pro/.qrc файлов и MOC. Всё — чистый Python. Ресурсы (шейдеры, иконки) загружаются обычным open() или через pyrcc6 если нужна упаковка.*

---

2. QObject и сигналы/слоты

2.1 Сигналы — pyqtSignal

В Qt C++ нужны Q_OBJECT + MOC. В PyQt6 — просто атрибут класса. Сигнал объявляется на уровне класса, не в __init__.

```
from PyQt6.QtCore import QObject, pyqtSignal, pyqtSlot
```

```
class Counter(QObject):
    # Объявление сигналов — НА УРОВНЕ КЛАССА, не в __init__
    valueChanged = pyqtSignal(int)          # сигнал с аргументом int
    dataReady    = pyqtSignal(str, float)   # несколько аргументов
    finished     = pyqtSignal()             # без аргументов

    def __init__(self, parent=None):
        super().__init__(parent)
        self._value = 0

    def increment(self):
        self._value += 1
        self.valueChanged.emit(self._value)  # emit сигнала
```

2.2 Подключение сигналов

```
counter = Counter()
label   = QLabel('0')

# Подключить к методу
counter.valueChanged.connect(label.setNum)

# Подключить к лямбде
counter.valueChanged.connect(lambda v: label.setText(f'Значение: {v}'))

# Подключить к декорированному слоту
@pyqtSlot(int)           # необязателен, но даёт проверку типов
def on_value_changed(value: int):
    print(f'Новое значение: {value}')

counter.valueChanged.connect(on_value_changed)

# Отключить
counter.valueChanged.disconnect(on_value_changed)
```

2.3 Типы соединений

Тип соединения	Описание
`Qt.ConnectionType.AutoConnection`	По умолчанию. Прямое если один поток, Queued если разные
`Qt.ConnectionType.DirectConnection`	Слот вызывается сразу (синхронно)
`Qt.ConnectionType.QueuedConnection`	Слот вызывается в потоке получателя через event loop
`Qt.ConnectionType.BlockingQueuedConnection`	Как Queued, но блокирует отправителя до завершения

```
from PyQt6.QtCore import Qt

# Явное указание типа соединения:
worker.finished.connect(self.on_done,
    Qt.ConnectionType.QueuedConnection)
```

3. Управление памятью

PyQt6 использует те же принципы что Qt C++: иерархия parent/child. При удалении родителя все дочерние объекты удаляются. Python GC + Qt ownership работают вместе.

```
# parent передаётся в конструктор
window  = QWidget()                        # нет parent — корневой
button  = QPushButton('OK', parent=window) # parent = window
label   = QLabel('Привет', window)          # тоже parent = window

# При del window → button и label тоже удалятся

# WA_DeleteOnClose — Qt удалит виджет при закрытии окна
window.setAttribute(Qt.WidgetAttribute.WA_DeleteOnClose)
```

Ситуация	Рекомендация
Виджеты в окне	Передать parent= — Qt управляет жизнью
QObject-сервисы	parent= главного окна или None + хранить в self
Временные данные	Обычные Python-переменные, GC сам уберёт
Поток (QThread)	Хранить в self, иначе GC удалит до завершения
Worker в потоке	НЕ передавать parent= если moveToThread()

---

4. Основные виджеты GUI

4.1 Точка входа

```
from PyQt6.QtWidgets import (QApplication, QMainWindow, QWidget,
    QPushButton, QLabel, QLineEdit, QSlider, QCheckBox,
    QTreeWidget, QTableWidget, QSplitter, QDockWidget,
    QFileDialog, QMessageBox, QProgressDialog)
from PyQt6.QtCore    import Qt, QSettings, QTimer, QThread, pyqtSignal
from PyQt6.QtGui     import QIcon, QMatrix4x4, QVector3D
from PyQt6.QtOpenGLWidgets import QOpenGLWidget
```

4.2 Основные виджеты

Класс	Назначение и сигнал
QPushButton	Кнопка. Сигнал: clicked()
QLabel	Текст или изображение (статичный)
QLineEdit	Поле ввода. Сигнал: textChanged(str)
QSlider	Ползунок. Сигнал: valueChanged(int)
QComboBox	Выпадающий список. Сигнал: currentIndexChanged(int)
QSpinBox	Числовой ввод. Сигнал: valueChanged(int)
QDoubleSpinBox	Числовой ввод (float)

QCheckBox	Флажок. Сигнал: toggled(bool)
QTreeWidget	Дерево. Сигнал: itemClicked(item, col)
QTableWidget	Таблица. Сигнал: cellClicked(row, col)
QListWidget	Список строк
QTextEdit	Многострочный редактор
QGroupBox	Группировка с заголовком
QTabWidget	Вкладки. Сигнал: currentChanged(int)
QSplitter	Разделитель с изменяемым размером
QScrollArea	Прокручиваемая область
QOpenGLWidget	Виджет для OpenGL рендеринга

---

5. Компоновка (Layouts)

Layouts автоматически управляют позицией и размером виджетов. Никогда не используй фиксированные координаты — только layouts.

5.1 Основные Layouts

Класс	Описание
QVBoxLayout	Вертикальный столбец (сверху вниз)
QHBoxLayout	Горизонтальная строка (слева направо)
QGridLayout	Сетка (row, col, rowSpan, colSpan)
QFormLayout	Форма: метка + поле
QStackedLayout	Стек — только один виджет видим

5.2 Пример компоновки

```
from PyQt6.QtWidgets import QWidget, QVBoxLayout, QHBoxLayout, QLabel, QLineEdit

central = QWidget()
vlay = QVBoxLayout(central)   # layout сразу на виджет

hlay = QHBoxLayout()
hlay.addWidget(QLabel('Имя:'))
hlay.addWidget(QLineEdit())

vlay.addLayout(hlay)
vlay.addStretch()             # пружина — займёт пустое место
vlay.addWidget(QPushButton('OK'))

# Отступы
vlay.setContentsMargins(10, 10, 10, 10)  # left, top, right, bottom
vlay.setSpacing(8)
```

5.3 QGridLayout

```
from PyQt6.QtWidgets import QGridLayout

grid = QGridLayout()
#              виджет    row  col  rowSpan  colSpan
grid.addWidget(label,     0,   0)
grid.addWidget(line_edit, 0,   1,    1,       2)   # 2 колонки
grid.setColumnStretch(1, 1)   # колонка 1 растягивается
```

---

6. QMainWindow — главное окно

QMainWindow — стандартная оболочка: menubar, toolbar, statusbar, dockwidgets, central widget.

```
from PyQt6.QtWidgets import (QMainWindow, QDockWidget, QTreeWidget,
    QTableWidget, QToolBar, QFileDialog)
from PyQt6.QtGui    import QAction, QKeySequence
from PyQt6.QtCore   import Qt, QSettings

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle('IFC Viewer')
        self.resize(1280, 720)

        # Центральный виджет — обязателен
        self.gl_widget = GLWidget(self)
        self.setCentralWidget(self.gl_widget)

        self._setup_menu()
        self._setup_docks()
        self._restore_state()

    def _setup_menu(self):
        file_menu = self.menuBar().addMenu('&Файл')

        open_act = QAction('&Открыть...', self)
        open_act.setShortcut(QKeySequence.StandardKey.Open)
        open_act.triggered.connect(self._open_file)
        file_menu.addAction(open_act)

        file_menu.addSeparator()
        file_menu.addAction('&Выход', self.close)

    def _setup_docks(self):
        # Дерево IFC слева
        dock_tree = QDockWidget('Структура IFC', self)
        self.tree = QTreeWidget()
        self.tree.setHeaderLabel('Элементы')
        dock_tree.setWidget(self.tree)
        self.addDockWidget(Qt.DockWidgetArea.LeftDockWidgetArea, dock_tree)

        # Свойства справа
        dock_props = QDockWidget('Свойства', self)
        self.props_table = QTableWidget(0, 2)
        self.props_table.setHorizontalHeaderLabels(['Свойство', 'Значение'])
        dock_props.setWidget(self.props_table)
        self.addDockWidget(Qt.DockWidgetArea.RightDockWidgetArea, dock_props)

    def _restore_state(self):
```

```
        s = QSettings('Company', 'IFCViewer')
        if s.value('geometry'):
            self.restoreGeometry(s.value('geometry'))

    def closeEvent(self, event):
        s = QSettings('Company', 'IFCViewer')
        s.setValue('geometry', self.saveGeometry())
        super().closeEvent(event)

    def _open_file(self):
        path, _ = QFileDialog.getOpenFileName(
            self, 'Открыть IFC', '',
            'IFC Files (*.ifc);;All Files (*)')
        if path:
            self.statusBar().showMessage(f'Загрузка: {path}')
```

7. OpenGL — QOpenGLWidget

QOpenGLWidget — основной способ встроить OpenGL рендеринг. Три виртуальных метода, PyOpenGL для GL-вызовов.

7.1 Структура класса

```
from PyQt6.QtOpenGLWidgets import QOpenGLWidget
from PyQt6.QtCore          import Qt, QTimer
from OpenGL.GL import *
import numpy as np

class GLWidget(QOpenGLWidget):
    def __init__(self, parent=None):
        super().__init__(parent)
        self._angle = 0.0

    def initializeGL(self):          # вызывается один раз
        glClearColor(0.1, 0.1, 0.15, 1.0)
        glEnable(GL_DEPTH_TEST)
        self._setup_shaders()
        self._setup_buffers()

    def resizeGL(self, w: int, h: int):  # при ресайзе
        glViewport(0, 0, w, h)
        # обновить матрицу проекции

    def paintGL(self):               # каждый кадр
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
        # draw calls...
```

7.2 Шейдеры — QOpenGLShaderProgram

```
from PyQt6.QtOpenGL import QOpenGLShaderProgram, QOpenGLShader

def _setup_shaders(self):
    self.program = QOpenGLShaderProgram(self)

    # Из файла (рекомендуется)
    self.program.addShaderFromSourceFile(
        QOpenGLShader.ShaderTypeBit.Vertex,   'shaders/vertex.glsl')
    self.program.addShaderFromSourceFile(
        QOpenGLShader.ShaderTypeBit.Fragment, 'shaders/fragment.glsl')
```

```
    # Или из строки
    vert_src = '''
        #version 330 core
        layout(location=0) in vec3 aPos;
        uniform mat4 mvp;
        void main() { gl_Position = mvp * vec4(aPos, 1.0); }
    '''
    self.program.addShaderFromSourceCode(
        QOpenGLShader.ShaderTypeBit.Vertex, vert_src)

    self.program.link()

# Использование в paintGL:
self.program.bind()
self.program.setUniformValue('mvp', mvp_matrix)  # QMatrix4x4
# draw...
self.program.release()
```

7.3 VAO и VBO через PyOpenGL

```
import numpy as np
from OpenGL.GL import *

def _setup_buffers(self):
    vertices = np.array([
        # x      y      z     nx    ny    nz
        -0.5,  -0.5,   0.0,  0.0,  0.0,  1.0,
         0.5,  -0.5,   0.0,  0.0,  0.0,  1.0,
         0.0,   0.5,   0.0,  0.0,  0.0,  1.0,
    ], dtype=np.float32)

    self.vao = glGenVertexArrays(1)
    self.vbo = glGenBuffers(1)

    glBindVertexArray(self.vao)
    glBindBuffer(GL_ARRAY_BUFFER, self.vbo)
    glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_STATIC_DRAW)

    stride = 6 * 4  # 6 float * 4 байт
    # Атрибут 0 — позиция
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(0))
    glEnableVertexAttribArray(0)
    # Атрибут 1 — нормаль
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE, stride, ctypes.c_void_p(12))
    glEnableVertexAttribArray(1)

    glBindVertexArray(0)

# Загрузка произвольного меша (точка входа для IFC-данных)
def load_mesh(self, vertices: np.ndarray, indices: np.ndarray):
    glBindVertexArray(self.vao)
    glBindBuffer(GL_ARRAY_BUFFER, self.vbo)
    glBufferData(GL_ARRAY_BUFFER, vertices.nbytes, vertices, GL_DYNAMIC_DRAW)
    # аналогично для EBO если есть indices
    self.index_count = len(indices)
    glBindVertexArray(0)
    self.update()  # перерисовать
```

7.4 QMatrix4x4 — матрицы для 3D

```
from PyQt6.QtGui import QMatrix4x4, QVector3D, QQuaternion

# Model матрица
model = QMatrix4x4()
```

```
model.setToIdentity()
model.translate(0.0, 0.0, -3.0)
model.rotate(self._angle, QVector3D(0, 1, 0))  # вращение по Y

# View матрица
view = QMatrix4x4()
view.lookAt(
    QVector3D(0, 0, 5),   # eye
    QVector3D(0, 0, 0),   # center
    QVector3D(0, 1, 0)    # up
)

# Projection матрица
proj = QMatrix4x4()
proj.perspective(45.0, self.width() / self.height(), 0.1, 1000.0)

mvp = proj * view * model
self.program.setUniformValue('mvp', mvp)
```

---

8. Обработка событий

Переопределяй виртуальные методы в своём виджете. В PyQt6 API событий немного отличается от C++.

```
from PyQt6.QtCore import Qt
from PyQt6.QtGui  import QMouseEvent, QWheelEvent, QKeyEvent

def mousePressEvent(self, event: QMouseEvent):
    if event.button() == Qt.MouseButton.LeftButton:
        self._last_pos = event.position().toPoint()  # .pos() в C++

def mouseMoveEvent(self, event: QMouseEvent):
    if event.buttons() & Qt.MouseButton.LeftButton:
        delta = event.position().toPoint() - self._last_pos
        self._last_pos = event.position().toPoint()
        self._camera.rotate(delta.x() * 0.3, delta.y() * 0.3)
        self.update()

def wheelEvent(self, event: QWheelEvent):
    delta = event.angleDelta().y() / 120.0  # +1 или -1 на шаг колеса
    self._camera.zoom(delta)
    self.update()

def keyPressEvent(self, event: QKeyEvent):
    if event.key() == Qt.Key.Key_Escape:
        self.close()
    elif event.key() == Qt.Key.Key_F:
        self._camera.fit_to_scene(self._bbox)
        self.update()
    elif event.key() == Qt.Key.Key_W:
        self._wireframe = not self._wireframe
        self.update()
```

> *Важное отличие от C++: event.pos() → event.position().toPoint(). В Qt6 position() возвращает QPointF (float), toPoint() конвертирует в QPoint.*

---

9. Таймеры

```
from PyQt6.QtCore import QTimer

# Разовый таймер
QTimer.singleShot(1000, self, lambda: print('Прошла 1 секунда'))

# Повторяющийся — анимация ~60 fps
self.timer = QTimer(self)
self.timer.timeout.connect(self._on_frame)
self.timer.start(16)    # 16 мс ≈ 60 fps

def _on_frame(self):
    self._angle += 1.0
    self.gl_widget.update()  # запросить перерисовку

self.timer.stop()       # остановить
```

---

10. Диалоги

```
from PyQt6.QtWidgets import QFileDialog, QMessageBox, QInputDialog

# Открыть файл
path, _ = QFileDialog.getOpenFileName(
    self, 'Открыть модель', '',
    '3D Files (*.ifc *.obj);;All (*)')

# Сохранить файл
path, _ = QFileDialog.getSaveFileName(self, 'Сохранить', '', '*.png')

# Выбрать папку
folder = QFileDialog.getExistingDirectory(self, 'Выбери папку')

# Сообщения
QMessageBox.information(self, 'Инфо', 'Загрузка завершена!')
QMessageBox.warning(self, 'Внимание', 'Файл не найден!')

btn = QMessageBox.question(self, 'Выход', 'Выйти?',
    QMessageBox.StandardButton.Yes | QMessageBox.StandardButton.No)
if btn == QMessageBox.StandardButton.Yes:
    self.close()

# Ввод числа
val, ok = QInputDialog.getDouble(self, 'FOV', 'Значение:', 45.0, 1.0, 179.0, 1)
if ok:
    self._fov = val
```

---

11. Многопоточность — QThread

Загрузка .ifc файлов может занять секунды. QThread + Worker — стандартный паттерн чтобы UI не зависал.

```
from PyQt6.QtCore import QThread, QObject, pyqtSignal

class IFCLoaderWorker(QObject):
```

```
    progress = pyqtSignal(int)       # 0-100
    finished = pyqtSignal(object)    # SceneData
    error    = pyqtSignal(str)

    def __init__(self, path: str):
        super().__init__()
        self._path = path
        self._cancelled = False

    def run(self):
        try:
            import ifcopenshell
            import ifcopenshell.geom
            model = ifcopenshell.open(self._path)
            products = model.by_type('IfcProduct')
            scene_data = []
            for i, product in enumerate(products):
                if self._cancelled:
                    return
                if product.Representation:
                    # получить геометрию...
                    scene_data.append(product)
                self.progress.emit(int(i / len(products) * 100))
            self.finished.emit(scene_data)
        except Exception as e:
            self.error.emit(str(e))

    def cancel(self):
        self._cancelled = True


# Запуск в MainWindow:
def _load_ifc(self, path: str):
    self._thread = QThread(self)
    self._worker = IFCLoaderWorker(path)
    self._worker.moveToThread(self._thread)

    self._thread.started.connect(self._worker.run)
    self._worker.finished.connect(self._on_loaded)
    self._worker.finished.connect(self._thread.quit)
    self._worker.error.connect(self._on_error)
    self._worker.progress.connect(self._progress_dlg.setValue)
    self._thread.finished.connect(self._worker.deleteLater)

    self._thread.start()

# НИКОГДА не трогать виджеты из worker-потока!
# Только через сигналы с QueuedConnection.
```

> *Важно хранить thread и worker в self (self._thread, self._worker). Если хранить в локальной переменной — Python GC удалит их до завершения потока.*

---

12. QSettings — настройки

```
from PyQt6.QtCore import QSettings

s = QSettings('MyCompany', 'IFCViewer')

# Сохранить
s.setValue('window/geometry',  self.saveGeometry())
s.setValue('window/state',     self.saveState())
```

```
s.setValue('render/fov',       45.0)
s.setValue('render/wireframe', False)
s.setValue('last_file',        '/path/to/file.ifc')

# Загрузить (второй аргумент — значение по умолчанию)
geom = s.value('window/geometry')
if geom: self.restoreGeometry(geom)

fov       = float(s.value('render/fov',       45.0))
wireframe = s.value('render/wireframe', False, type=bool)
last_file = s.value('last_file', '')
```

---

13. ifcopenshell — работа с IFC

ifcopenshell — Python-библиотека для парсинга .ifc файлов. Извлечение геометрии и метаданных.

13.1 Открытие и навигация

```
import ifcopenshell

model = ifcopenshell.open('building.ifc')

# Получить все элементы типа
walls   = model.by_type('IfcWall')
windows = model.by_type('IfcWindow')
floors  = model.by_type('IfcBuildingStorey')

# Конкретный элемент по GlobalId
element = model.by_guid('2O2Fr$t4X7Zf8NOew3FLOH')

# Иерархия
project  = model.by_type('IfcProject')[0]
for site in project.IsDecomposedBy[0].RelatedObjects:
    for building in site.IsDecomposedBy[0].RelatedObjects:
        for storey in building.IsDecomposedBy[0].RelatedObjects:
            print(f'Этаж: {storey.Name}')
```

13.2 Свойства элементов (PropertySet)

```
def get_properties(element) -> dict:
    props = {}
    for rel in element.IsDefinedBy:
        if rel.is_a('IfcRelDefinesByProperties'):
            pset = rel.RelatingPropertyDefinition
            if pset.is_a('IfcPropertySet'):
                for prop in pset.HasProperties:
                    if prop.is_a('IfcPropertySingleValue'):
                        props[prop.Name] = prop.NominalValue.wrappedValue
    return props

# Пример:
wall = model.by_type('IfcWall')[0]
print(get_properties(wall))
# {'LoadBearing': True, 'IsExternal': False, 'FireRating': '2h'}
```

13.3 Геометрия

```
import ifcopenshell.geom
import numpy as np
```

```
settings = ifcopenshell.geom.settings()
settings.set(settings.USE_WORLD_COORDS, True)  # мировые координаты

for product in model.by_type('IfcProduct'):
    if not product.Representation:
        continue
    try:
        shape = ifcopenshell.geom.create_shape(settings, product)
        g = shape.geometry

        verts   = np.array(g.verts).reshape(-1, 3)     # позиции XYZ
        normals = np.array(g.normals).reshape(-1, 3)   # нормали
        faces   = np.array(g.faces)                    # индексы треугольников

        # Объединить в один массив для VAO:
        # stride: [x, y, z, nx, ny, nz]
        vertex_data = np.hstack([verts, normals]).astype(np.float32)

        # → передать в GLWidget.load_mesh(vertex_data, faces)

    except Exception:
        pass  # не все элементы имеют геометрию
```

IFC использует Z-up координаты, OpenGL — Y-up. Нужна матрица поворота: model.rotate(-90, QVector3D(1,0,0)) при рендере, или конвертировать координаты при загрузке.

---

14. Базовые типы данных

Тип PyQt6	Описание и аналог
QString → str	В PyQt6 нет QString — используется обычный Python str
QList → list	Обычный Python list
QVector3D	3D вектор. from PyQt6.QtGui import QVector3D
QVector2D	2D вектор
QMatrix4x4	Матрица 4x4 для трансформаций
QQuaternion	Кватернион для вращений
QColor	QColor(255,128,0) или QColor('#ff8000')
QPoint / QPointF	Точка (int / float). В Qt6 события дают QPointF
QSize / QSizeF	Размер (ширина × высота)
QRect / QRectF	Прямоугольник
QImage / QPixmap	Изображение (CPU / GPU-оптимизировано)
QVariant → object	В PyQt6 не нужен — Python динамически типизирован
np.ndarray	Для данных OpenGL (вершины, индексы) — numpy обязателен

---

15. Отладка и утилиты

15.1 qDebug — аналог в Python

```
# Простой print — работает, но без уровней
print(f'Объект: {element.Name}')

# Через logging — рекомендуется для продакшена
import logging
logging.basicConfig(level=logging.DEBUG)
log = logging.getLogger(__name__)

log.debug('Загрузка начата')
log.warning('Файл без геометрии: %s', element.GlobalId)
log.error('Ошибка парсинга: %s', str(e))
```

15.2 Работа с файлами

```
from pathlib import Path

path = Path('model.ifc')
if path.exists():
    text = path.read_text(encoding='utf-8')
    lines = text.splitlines()

# Запись
Path('output.txt').write_text('данные', encoding='utf-8')
```

15.3 Упаковка — PyInstaller

```
pip install pyinstaller

# Сборка в один .exe
pyinstaller --onefile --windowed main.py

# С иконкой и дополнительными файлами
pyinstaller --onefile --windowed \
    --icon=resources/icon.ico \
    --add-data 'shaders;shaders' \
    main.py

# Результат в dist/main.exe
# Протестировать на чистой машине без Python!
```

Стек для IFC Viewer на PyQt6

Задача	Инструмент
Окно / GUI / Layouts	QMainWindow + QWidget + Layouts + QDockWidget
OpenGL рендеринг	QOpenGLWidget + PyOpenGL + numpy
Шейдеры	QOpenGLShaderProgram + .glsl файлы
Геометрия VAO/VBO	glGenVertexArrays + glGenBuffers + numpy.array
Матрицы / векторы	QMatrix4x4, QVector3D, QQuaternion
Управление камерой	mouseMoveEvent + wheelEvent + QMatrix4x4
Парсинг .ifc	ifcopenshell + ifcopenshell.geom

Загрузка в фоне	QThread + Worker + pyqtSignal
Анимация / кадры	QTimer(16мс) + update()
Настройки	QSettings
Диалоги	QFileDialog, QMessageBox, QInputDialog
Упаковка	PyInstaller --onefile --windowed

Python 3.10+ · PyQt6 6.x · PyOpenGL 3.x · ifcopenshell 0.7+